walrus-core 크레이트 상세 분석

개요

`walrus-core` 크레이트는 Walrus 탈중앙화 블롭 스토리지 시스템의 핵심 기능을 담당하는 라이브러리입니다. 이 크레이트는 RedStuff 인코딩 알고리즘, 머클 트리 구현, 비잔틴 장애 허용 로직, 그리고 핵심 데이터 구조들을 제공합니다.

주요 컴포넌트

1. RedStuff 인코딩 알고리즘 (encoding 모듈)

RedStuff는 Walrus에서 사용하는 2차원 Reed-Solomon 기반 인코딩 알고리즘입니다. 이 알고리즘은 데이터를 여러 스토리지 노드에 분산 저장하면서도 일부 노드의 장애나 비잔틴 행동에도 불구하고 데이터 복구가 가능하도록 설계되었습니다.

1.1 인코딩 구조

```
// 인코딩 축 정의
pub trait EncodingAxis {
const IS_PRIMARY: bool;
}

pub struct Primary;
impl EncodingAxis for Primary {
const IS_PRIMARY: bool = true;
```

```
}

pub struct Secondary;
impl EncodingAxis for Secondary {
const IS_PRIMARY: bool = false;
}
```

RedStuff는 2차원 인코딩을 사용하여:

- **Primary 축**: 행(row) 방향 인코딩
- **Secondary 축**: 열(column) 방향 인코딩

1.2 Reed-Solomon 인코딩 구현

```
pub struct ReedSolomonEncoder {
encoder: reed_solomon_simd::ReedSolomonEncoder,
n_source_symbols: NonZeroU16, // 소스 심볼 수
n_shards: NonZeroU16, // 총 샤드 수
symbol_size: NonZeroU16, // 심볼 크기
source_symbols: Vec<Vec<u8>>, // 소스 심볼들
}
```

핵심 특징:

- SIMD 최적화된 Reed-Solomon 구현 사용
- 최대 65,535개의 소스 심볼 지원 (`MAX_SOURCE_SYMBOLS`)
- 2바이트 정렬 요구사항

- 비잔틴 장애 허용을 위한 중복도 제공

1.3 인코딩 설정

```
pub struct ReedSolomonEncodingConfig {
pub n_shards: NonZeroU16, // 총 샤드 수 (N)
pub n_primary_source_symbols: NonZeroU16, // 1차 소스 심볼 수
pub n_secondary_source_symbols: NonZeroU16, // 2차 소스 심볼 수
pub symbol_size: NonZeroU16, // 심볼 크기
}
```

매개변수 관계:

- `N = n_shards` : 총 샤드 수
- `f = max_n_faulty(N)` : 최대 비잔틴 노드 수 = ⌊(N-1)/3⌋
- Primary 소스 심볼: `N - 2f - σ` (σ는 안전 마진)
- Secondary 소스 심볼: `N - f - σ`

2. 슬리버 (Sliver) 데이터 구조

슬리버는 인코딩된 블롭의 기본 저장 단위입니다.

2.1 슬리버 정의

```
pub struct SliverData<T: EncodingAxis> {
pub symbols: Symbols, // 인코딩된 심볼들
pub index: SliverIndex, // 슬리버 인덱스
_sliver_type: PhantomData<T>, // 타입 마커
```

```
}

// 타입 별칭
pub type PrimarySliver = SliverData<Primary>;
pub type SecondarySliver = SliverData<Secondary>;
pub type Sliver = ByAxis<PrimarySliver, SecondarySliver>;
```

2.2 슬리버 검증

```
impl<T: EncodingAxis> SliverData<T> {
pub fn verify(
&self,
encoding_config: &EncodingConfig,
metadata: &BlobMetadata,
) -> Result<(), SliverVerificationError> {
// 1. 슬리버 길이 검증
// 2. 심볼 크기 검증
// 3. 머클 루트 검증
}
}
```

검증 과정:

1. 슬리버의 심볼 수와 크기가 메타데이터와 일치하는지 확인

2. 슬리버의 머클 루트가 메타데이터의 해시와 일치하는지 확인

3. 인코딩 설정과 호환되는지 확인

3. 머클 트리 구현 (merkle.rs)

머클 트리는 슬리버의 무결성을 보장하고 효율적인 인증을 제공합니다.

3.1 노드 구조

```
#[derive(Clone, PartialEq, Eq, Debug, Deserialize, Serialize)]
pub enum Node {
Empty, // 빈 서브트리
Digest([u8; DIGEST_LEN]), // 32바이트 해시값
}

pub const DIGEST_LEN: usize = 32;
const LEAF_PREFIX: [u8; 1] = [0]; // 리프 노드 접두사
const INNER_PREFIX: [u8; 1] = [1]; // 내부 노드 접두사
```

3.2 머클 증명

```
pub trait MerkleAuth: Clone + Debug {
fn verify_proof(
&self,
root: &Node,
leaf_count: usize,
```

```
leaf: &[u8],
leaf_index: usize,
) -> Result<(), MerkleProofError>;
fn compute_root(
&self,
leaf: &[u8],
leaf_index: usize
) -> Result<Node, MerkleProofError>;
}
```

머클 트리 특징:

- Blake2b-256 해시 함수 사용
- 리프와 내부 노드 구분을 위한 접두사 사용
- 빈 노드에 대한 최적화된 처리
- 효율적인 증명 검증

3.3 슬리버 인증 메커니즘

```
impl<T: EncodingAxis> SliverData<T> {
pub fn get_merkle_root<H: HashFunction<DIGEST_LEN>>(
&self,
config: &EncodingConfigEnum,
) -> Result<Node, RecoverySymbolError> {
let tree = MerkleTree::<H>::from_leaves(
```

```
self.symbols.iter().map(|symbol| symbol.as_ref())
)?;
Ok(tree.root())
}
}
```

인증 과정:

1. 슬리버의 각 심볼을 리프로 하는 머클 트리 구성
2. 트리의 루트 해시 계산
3. 메타데이터의 해시와 비교하여 무결성 검증

4. 비잔틴 장애 허용 (bft.rs)

BFT 모듈은 비잔틴 장애 허용 시스템의 핵심 매개변수를 계산합니다.

4.1 BFT 매개변수

```
/// n개 컴포넌트를 가진 BFT 시스템에서 최대 장애 노드 수
#[inline]
pub fn max_n_faulty(n: NonZeroU16) -> u16 {
(n.get() - 1) / 3
}

/// n개 컴포넌트를 가진 BFT 시스템에서 최소 정상 노드 수
#[inline]
pub fn min_n_correct(n: NonZeroU16) -> NonZeroU16 {
```

```
(n.get() - max_n_faulty(n))
.try_into()
.expect("max_n_faulty < n")
}
```

BFT 안전성 보장:

- 총 노드 수 `n` 에 대해 최대 `f = ⌊(n-1)/3⌋` 개의 비잔틴 노드 허용
- 최소 `2f + 1` 개의 정상 노드 필요
- 데이터 복구를 위해 `n - f` 개 이상의 슬리버 필요

4.2 안전성 분석

수학적 기반:

- 비잔틴 장애 허용의 고전적인 `3f + 1` 공식 적용
- `f < n/3` 조건을 만족해야 안전성 보장
- 복구 임계값: Primary는 `n - 2f` , Secondary는 `n - f`

5. 핵심 데이터 구조

5.1 BlobId

```
#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Serialize, Deserialize, Ha
pub struct BlobId(pub [u8; 32]);

impl BlobId {
pub fn from_metadata(
```

```
merkle_root: Node,
encoding: EncodingType,
unencoded_length: u64
) -> Self {
let mut hasher = Blake2b256::default();
hasher.update([encoding.into()]);
hasher.update(unencoded_length.to_le_bytes());
hasher.update(merkle_root.bytes());
Self(hasher.finalize().into())
}
}
```

BlobId 생성:

1. 인코딩 타입 (1바이트)
2. 원본 데이터 길이 (8바이트, little-endian)
3. 머클 루트 (32바이트)
4. 위 데이터의 Blake2b-256 해시가 BlobId

5.2 심볼 식별자

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct SymbolId {
primary: SliverIndex, // Primary 슬리버 인덱스
secondary: SliverIndex, // Secondary 슬리버 인덱스
```

```
}
```

심볼 좌표계:

- 2차원 격자에서 각 심볼의 위치를 (primary, secondary) 좌표로 표현
- 복구 시 필요한 심볼들의 위치 추적에 사용

5.3 인코딩 타입

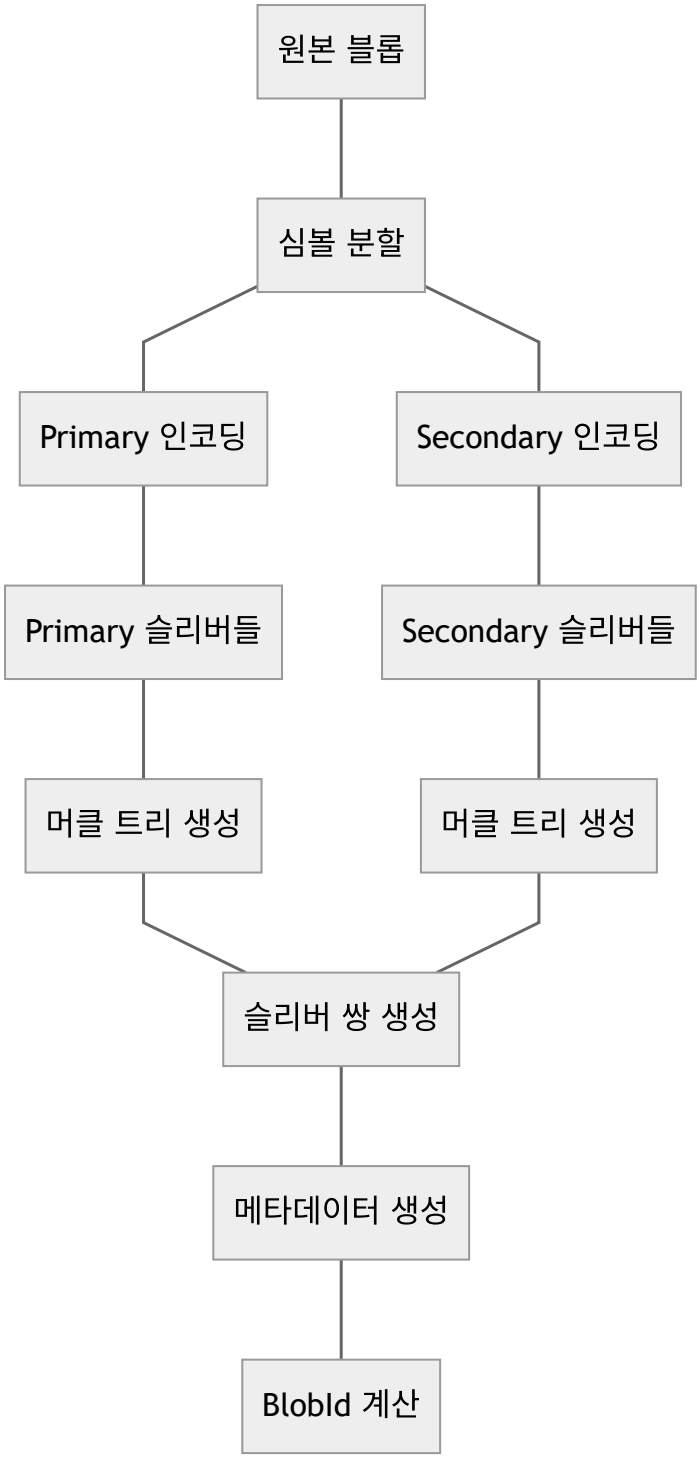
```
#[derive(Debug, PartialEq, Eq, PartialOrd, Ord, Clone, Copy, Hash)]
#[repr(u8)]
pub  enum EncodingType {
#[deprecated] // 더 이상 사용되지 않음
RedStuffRaptorQ = 0,
RS2 = 1, // Reed-Solomon 기반 RedStuff
}
```

현재 지원 인코딩:

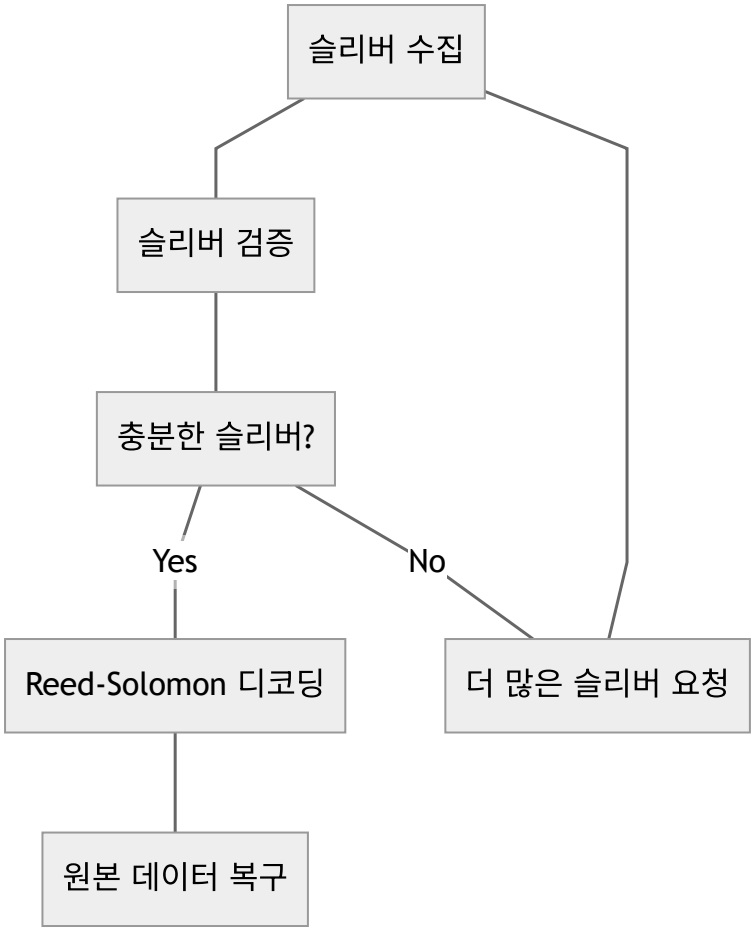
- `RS2` : Reed-Solomon 기반 RedStuff (현재 기본값)
- 2바이트 정렬 요구사항
- 최대 심볼 크기: 65,534바이트

데이터 흐름

1. 인코딩 과정


원본 블롭
심볼 분할
Primary 인코딩
Secondary 인코딩
Primary 슬리버들
Secondary 슬리버들
머클 트리 생성
머클 트리 생성
슬리버 쌍 생성
메타데이터 생성
BlobId 계산

2. 디코딩 과정



성능 최적화

1. SIMD 가속화

```
// Reed-Solomon 연산에 SIMD 명령어 활용
use reed_solomon_simd::ReedSolomonEncoder;

// AVX2/SSSE3 (x86) 및 Neon (ARM) 지원
// 벡터 연산으로 인코딩/디코딩 성능 향상
```

2. 메모리 최적화

- 심볼 정렬: 2바이트 경계 정렬로 SIMD 효율성 향상

- 지연 평가: 필요시에만 머클 트리 계산
- 제로 카피: 가능한 경우 데이터 복사 최소화

3. 병렬 처리

```
// 슬리버별 독립적인 처리 가능
impl SliverData<T> {
// 각 슬리버의 머클 트리를 병렬로 계산
pub fn compute_merkle_root_parallel(&self) -> Node {
// 병렬 처리 로직
}
}
```

보안 고려사항

1. 암호학적 해시

- Blake2b-256: 빠르고 안전한 해시 함수
- 머클 트리의 도메인 분리: 리프/내부 노드 접두사 사용
- 충돌 저항성과 무결성 보장

2. 비잔틴 장애 허용

- 수학적으로 증명된 `3f + 1` 임계값 사용
- 악의적 노드의 잘못된 데이터 탐지 가능
- 복구 임계값을 통한 가용성 보장

3. 타입 안전성

```
// 컴파일 타임 타입 검사로 버그 방지
pub struct SliverData<T: EncodingAxis> {
_sliver_type: PhantomData<T>, // 타입 안전성 보장
}

// Primary/Secondary 슬리버 혼동 방지
```

에러 처리

1. 인코딩 에러

```
#[derive(Debug, Error)]
pub enum EncodeError {
#[error("데이터가 너무 큼")]
DataTooLarge(#[from] DataTooLargeError),
#[error("잘못된 데이터 크기")]
InvalidDataSize(#[from] InvalidDataSizeError),
}
```

2. 검증 에러

```
#[derive(Debug, Error)]
```

```rust
pub enum SliverVerificationError {
#[error("슬리버 길이 불일치")]
LengthMismatch,
#[error("머클 루트 불일치")]
MerkleRootMismatch,
#[error("심볼 크기 불일치")]
SymbolSizeMismatch,
}
```

테스트 및 검증

1. 단위 테스트

```rust
#[cfg(test)]
mod tests {
#[test]
fn test_encoding_roundtrip() {
// 인코딩-디코딩 라운드트립 테스트
let data = b"Hello, Walrus!";
let config = ReedSolomonEncodingConfig::new(7, 3, 4, 1024);
let encoded = config.encode_with_metadata(data).unwrap();
let decoded = decode_from_slivers(&encoded.0).unwrap();
assert_eq!(data, decoded.as_slice());
```

```
}

}
```

2. 속성 기반 테스트

- 임의의 데이터 크기와 인코딩 매개변수로 테스트
- 비잔틴 노드 시뮬레이션
- 부분 슬리버 손실 시나리오

결론

`walrus-core` 크레이트는 Walrus 시스템의 핵심 기능을 제공하는 잘 설계된 라이브러리입니다. RedStuff 인코딩 알고리즘, 머클 트리 기반 인증, 그리고 비잔틴 장애 허용 메커니즘이 유기적으로 결합되어 안전하고 효율적인 분산 스토리지 시스템을 구현합니다.

주요 강점:

- 수학적으로 검증된 BFT 안전성
- SIMD 최적화된 고성능 인코딩
- 타입 안전한 Rust 구현
- 포괄적인 에러 처리
- 확장 가능한 아키텍처

이러한 설계는 Walrus가 대규모 분산 환경에서도 안정적으로 동작할 수 있는 기반을 제공합니다.